

TÓPICOS ESPECIAIS EM SOFTWARE

INTRODUÇÃO AO FLASK

O QUE É UMA APLICAÇÃO WEB?

- É um programa acessado por meio de um navegador.
- O navegador atua como cliente e solicita recursos a um servidor.
- O servidor processa a solicitação e devolve uma resposta.
- Python pode executar a lógica do servidor.
- HTML estrutura a página apresentada ao usuário.

CLIENTE E SERVIDOR

1. O usuário acessa um endereço no navegador.
2. O navegador envia uma requisição HTTP.
3. O servidor recebe a requisição.
4. O programa Python processa a solicitação.
5. O servidor devolve uma resposta.
6. O navegador exibe o conteúdo recebido.

O QUE É FLASK?

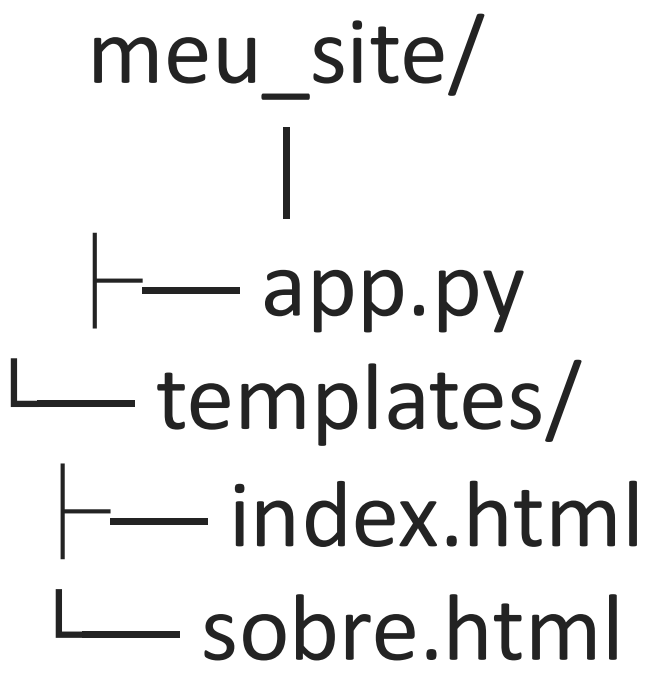
- Flask é um framework Web para Python.
- Um framework fornece recursos que facilitam a criação de aplicações.
- Flask permite começar com poucos comandos.
- É adequado para estudar rotas, requisições, templates e formulários.
- Nesta disciplina, será usado para compreender os fundamentos do desenvolvimento Web.

PREPARANDO O AMBIENTE

```
# No terminal do projeto:
pip install flask

# Para verificar a instalação:
pip show flask
```

ESTRUTURA INICIAL DO PROJETO



PRIMEIRA APLICAÇÃO FLASK

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def inicio():
    return "Olá! Minha primeira aplicação Web com Python."

if __name__ == "__main__":
    app.run(debug=True)
```

ENTENDENDO O CÓDIGO

- from flask import Flask: importa a classe Flask.
- app = Flask(__name__): cria a aplicação.
- @app.route('/'): associa um endereço a uma função Python.
- return: define o conteúdo enviado ao navegador.
- app.run(): inicia o servidor de desenvolvimento.
- debug=True: facilita os testes durante o desenvolvimento.

EXECUTANDO A APLICAÇÃO

```
python app.py

# Depois, abra no navegador o endereço mostrado
# pelo Flask, normalmente:
# http://127.0.0.1:5000/
```

- O servidor precisa permanecer em execução enquanto a aplicação estiver sendo acessada.

CRIANDO MAIS DE UMA ROTA

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def inicio():
    return "Página inicial"

@app.route("/sobre")
def sobre():
    return "Página sobre o sistema"

if __name__ == "__main__":
    app.run(debug=True)
```

O QUE SÃO ROTAS?

- Uma rota representa um endereço da aplicação.
- `/` representa a página inicial.
- `/sobre` representa outra página.
- Cada rota pode executar uma função Python diferente.
- A função associada à rota é chamada quando o navegador acessa aquele endereço.

ROTAS DINÂMICAS

```
from flask import Flask

app = Flask(__name__)

@app.route("/usuario/<nome>")
def usuario(nome):
    return f"Olá, {nome}!"

if __name__ == "__main__":
    app.run(debug=True)
```

HTML DIRETAMENTO DO PYTHON

- É possível retornar HTML diretamente em uma rota.
- Entretanto, misturar HTML grande com Python dificulta a organização.
- A solução é separar as páginas em arquivos HTML.
- No Flask, esses arquivos normalmente ficam na pasta `templates`.

USANDO RENDER_TEMPLATE()

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def inicio():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)
```

ARQUIVO TEMPLATES/INDEX.HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>Meu Site</title>
</head>
<body>
    <h1>Desenvolvimento Web com Python</h1>
    <p>Esta página foi carregada pelo Flask.</p>
</body>
</html>
```

PYTHON ENVIANDO DADOS PARA O HTML

```
@app.route("/usuario/<nome>")
def usuario(nome):
    return render_template(
        "usuario.html",
        nome_usuario=nome
    )
```

JINJA: EXIBINDO UMA VARIÁVEL

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>Usuário</title>
</head>
<body>
    <h1>Olá, {{ nome_usuario }}!</h1>
    <p>Bem-vindo ao sistema.</p>
</body>
</html>
```

O QUE SIGNIFICA {{ ... }}?

- Flask utiliza o mecanismo de templates Jinja.
- `{{ ... }}` é usado para exibir valores.
- O valor é fornecido pelo Python através de `render_template()`.
- Assim, a mesma página HTML pode apresentar conteúdos diferentes.

EXEMPLO COM UMA LISTA

```
@app.route("/alunos")
def alunos():
    nomes = ["Ana", "Carlos", "Maria", "Pedro"]

return render_template(
        "alunos.html",
        lista=nomes
    )
```

PERCORRENDO A LISTA NO HTML

```
<h1>Alunos cadastrados</h1>

<ul>
{% for nome in lista %}
    <li>{{ nome }}</li>
{% endfor %}
</ul>
```

- {% ... %} é usado para comandos do template, como repetição e decisão.

GET: SOLICITANDO UMA PÁGINA

- Quando digitamos um endereço no navegador, normalmente fazemos uma requisição GET.
- GET é utilizado para solicitar/consultar um recurso.
- Exemplo: acessar `/alunos` solicita ao servidor a página de alunos.
- Agora veremos um formulário simples enviando dados por GET.

PRIMEIRO FORMULÁRIO

```
<form action="/saudacao" method="GET">
<label>Nome:</label>
<input type="text" name="nome">

<button type="submit">Enviar</button>
</form>
```

- *O atributo name identifica o dado que será enviado ao servidor.*

RECEBENDO O DADO DO FORMULÁRIO

```
from flask import request

@app.route("/saudacao")
def saudacao():
    nome = request.args.get("nome")

    return render_template(
        "saudacao.html",
        nome=nome
    )
```

- *request.args.get('nome') recupera o valor enviado pelo formulário via GET.*

APLICAÇÃO COMPLETA – APP.PY

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/")
def inicio():
    return render_template("index.html")

@app.route("/saudacao")
def saudacao():
    nome = request.args.get("nome")
    return render_template("saudacao.html", nome=nome)

if __name__ == "__main__":
    app.run(debug=True)
```

APLICAÇÃO COMPLETA – INDEX.HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>Cadastro</title>
</head>
<body>
    <h1>Informe seu nome</h1>
    <form action="/saudacao" method="GET">
        <input type="text" name="nome">
        <button type="submit">Enviar</button>
    </form>
</body>
</html>
```

APLICAÇÃO COMPLETA – SAUDACAO.HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>Saudação</title>
</head>
<body>
    <h1>Olá, {{ nome }}!</h1>
    <a href="/">Voltar</a>
</body>
</html>
```

EXERCÍCIO

- Desenvolva uma aplicação Web utilizando **Python, Flask e HTML** para calcular a média de um aluno.
- A página inicial deverá apresentar um formulário solicitando:
  - Nome do aluno;
  - Nota 1;
  - Nota 2.
- Ao clicar no botão **Calcular**, os dados deverão ser enviados para uma rota chamada /resultado.
- Na aplicação Python, o programa deverá:
  - Receber os três valores enviados pelo formulário.
  - Converter as notas para valores numéricos.
  - Calcular a média:

EXERCÍCIO

- Enviar para uma segunda página HTML:nome do aluno;
  - primeira nota;
  - segunda nota;
  - média calculada.

